



Tecnológico de Monterrey

Campus Querétaro

Evidencia Implementación de Análisis Léxico (Automata y Expresión Regular)

Inaki Mancera Llano - A01708827

Materia: Implementacion de Metodos Computacionales

Fecha de entrega
28 de Mayo 2026

1. Contexto del lenguaje.

El lenguaje seleccionado fue etiquetas relacionadas con el texto HTML. HTML es un lenguaje utilizado para estructurar contenido en páginas web. Sus elementos se escriben mediante etiquetas rodeadas por los símbolos < y > para distinguirse del texto normal.

Para esta evidencia, no se analizará todo HTML, sino solamente un subconjunto de etiquetas relacionadas con texto. Esto permite construir un lenguaje regular más manejable y representarlo mediante un autómata finito determinista. Un lenguaje formal es un conjunto de cadenas sobre un alfabeto finito y un DFA es una máquina con estados conectados por arcos etiquetados, donde cada símbolo leído determina un cambio de estado.

Se van a aceptar etiquetas de HTML de texto con la forma <etiqueta> y </etiqueta>, siendo que en HTML la primera se utiliza para empezar su función específica y la segunda para marcar el final de dicha función o formato. En donde **etiqueta** puede ser:

$T = \{p, h1, h2, h3, h4, h5, h6, strong, em, span, mark, small, sub, sup\}$

Teniendo en cuenta lo anterior, el alfabeto para este lenguaje sería el siguiente:

$\Sigma = \{ <, >, /, p, h, 1, 2, 3, 4, 5, 6, s, t, r, o, n, g, e, m, a, k, l, u, b \}$

Por esto, el lenguaje se puede describir como el conjunto de cadenas formadas por una etiqueta de apertura <x> y una etiqueta de cierre </x> en donde **x** pertenece al conjunto de etiquetas válidas **T**.

Ejemplos de cadenas que pertenecen al lenguaje:

- <p>
- </p>
- <h1>
- </h6>
-
-
-
-
- <small>

Ejemplos de cadenas que no pertenecen al lenguaje:

- <div>
- <h7>
- <p
- p>
-
-

2. Expresión regular equivalente.

Una expresión regular es una forma compacta de describir un patrón dentro de una cadena de texto. Las expresiones regulares se relacionan con los autómatas porque ambos pueden describir lenguajes regulares.

Usando los caracteres especiales de expresiones regulares, la expresión regular propuesta para esta automata seria:

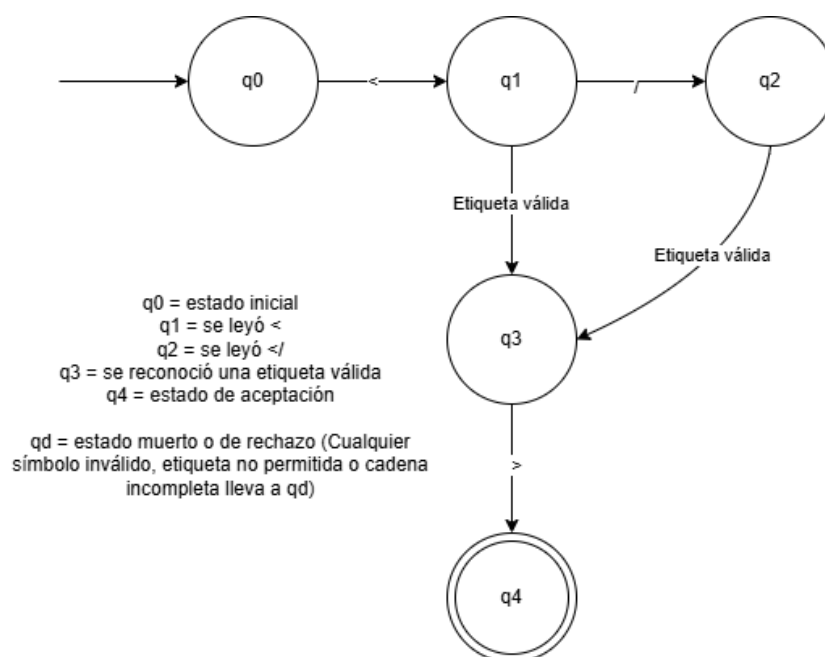
$\wedge < /? (p|h[1 - 6]|strong|em|span|mark|small|sub|sup) > \$$

La cual reconoce el lenguaje definido, solo acepta las etiquetas definidas en **T** y no se sale del alfabeto. En la que el símbolo $\wedge$ indica que no se puede empezar con otro carácter que no sea $<$ y el símbolo $\$$ indica que no se puede acabar con otro carácter que no sea $>$, mientras que en el medio están las posibilidades de etiquetas y la posibilidad de que sea una de cierre con el símbolo $?$ junto al carácter $/$.

3. Propuesta de autómata.

Para que el diagrama no sea demasiado complicado, se representará un DFA por etapas. El DFA reconoce la estructura general:

$< + /opcional + etiqueta\ válida + >$



4. Análisis de complejidad.

4.1 DFA en Prolog:

La implementación en Prolog procesa la entrada símbolo por símbolo. En cada paso, el programa intenta encontrar una transición válida desde el estado actual con el símbolo leído.

Sea n la cantidad de símbolos en la lista de entrada, como cada símbolo de la lista se consume una sola vez, la complejidad temporal es $O(n)$. Por esto el tiempo de ejecución crece linealmente con el número de símbolos de entrada.

Como la implementación usa recursión y cada llamada recursiva consume un símbolo, en el peor caso hay tantas llamadas como el tamaño de la lista de entrada. Por esto la complejidad espacial es $O(n)$.

4.2 Propuesta expresion regular:

En una implementación basada en expresión regular, el motor de regex revisara si la cadena completa coincide con el patrón.

Sea n la longitud de la cadena de entrada, como el conjunto de etiquetas válidas es finito y el patrón no tiene ambigüedades, la confirmación se puede hacer linealmente respecto a la longitud de la cadena. Por esto la complejidad temporal es $O(n)$.

Al regex solo ocupar confirmar si la cadena coincide con el patrón sin ocupar recorrer caracter por caracter, la complejidad espacial es $O(1)$.

5. Comparación.

Aspecto	DFA en Prolog	Expresión regular
Modelo	Automata finito determinista	Expresión regular equivalente
Implementación	Movimiento entre hechos y recursión	Motor de expresiones regulares
Entrada	Lista de símbolos	String
Complejidad temporal	$O(n)$	$O(n)$
Complejidad espacial	$O(n)$	$O(1)$
Ventaja	Muestra estados y transiciones	Es compacta y sencilla
Desventaja	Requiere de mas codigo	No muestra el proceso

En términos generales, ambas soluciones son lineales respecto al tamaño de su entrada. La principal diferencia es que la solución en Prolog opera sobre una lista de símbolos ya separados, mientras que la expresión regular opera directamente sobre un string.